





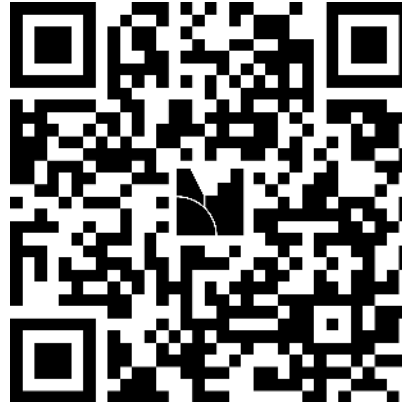
Stop Fighting Your Spring Boot Tests

Patterns für zuverlässige & schnelle Builds

Nürnberger Versicherung 03.12.2025

Philip Riecks - [PragmaTech GmbH](#) - [@rieckpil](#)

Interaktive Teilnahme

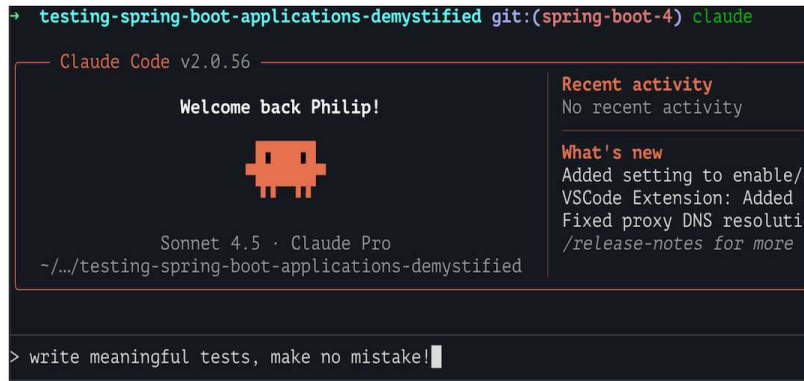


Gehe auf menti.com und verwende den Code **5490 0636**, um **anonym** Antworten zu den Quizfragen einzureichen und während des Vortrags Fragen zu stellen.

Starte mit den ersten beiden Fragen:

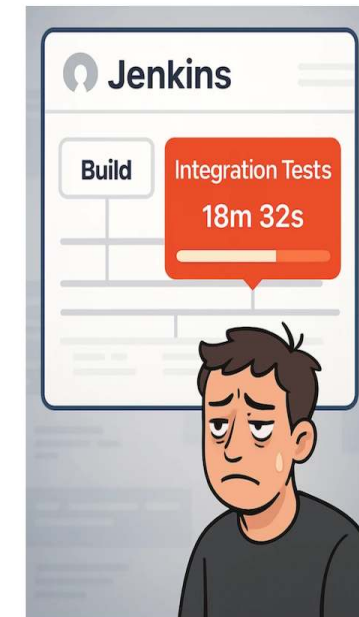
- Schreibst du deine Tests trotz LLMs und Code Agents noch von Hand?
- Macht dir das Schreiben von automatisierten Tests Spaß?

Spring Boot Testing - The Bad & Ugly



```
@Test
// TODO: Fix test
@Disabled("Flaky test, failing randomly on CI")
void contextLoads(@Autowired WebTestClient webTestClient) {
```

```
Caused by: org.springframework.beans.factory.UnsatisfiedDependencyException:
  Error creating bean with name 'publicController' defined in file [/home/riECKpil/development/c
  Unsatisfied dependency expressed through constructor parameter 0; nested exception is org.spri
  No qualifying bean of type 'de.riECKpil.learning.UserService' available:
  expected at least 1 bean which qualifies as autowire candidate. Dependency annotations: {}
  ... 68 more
Caused by: org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type 'de.riECKpil.learning.UserService' available:
  expected at least 1 bean which qualifies as autowire candidate. Dependency annotations: {}
  ... 87 more
```



```
→ testing-spring-boot-applications-demystified git:(spring-boot-4) ./mvnw -DskipTests package
```



Spring Boot Testing - The Good

```
$ ./mvnw verify

[INFO] -----
[INFO]  T E S T S
[INFO] -----
[INFO] Running com.example.UserServiceTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.1s
[INFO] Running com.example.PaymentControllerTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.4s
[INFO] Running com.example.OrderProcessingTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.7s
[INFO] Running com.example.DatabaseIntegrationTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.2s

[INFO] Results:
[INFO] Tests run: 247, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 04:15 min
[INFO] Finished at: 2025-01-15T10:46:35+01:00
```

kürzere Feedbackzyklen

Continuous Deployment

Refactoring
ohne Regressionen

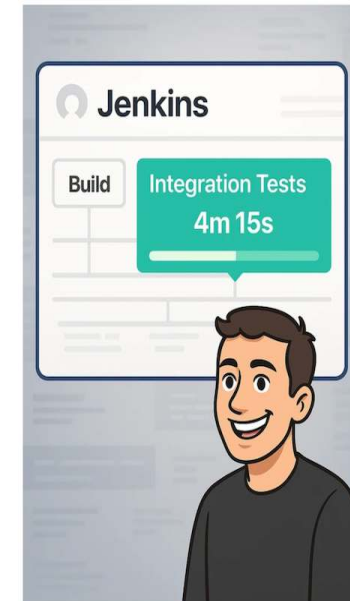
(kürzere Kaffeepausen)

Tests als Dokumentation

Tests als Sicherheitsnetz

schnelles Deployment

unbekannten Code schneller verstehen



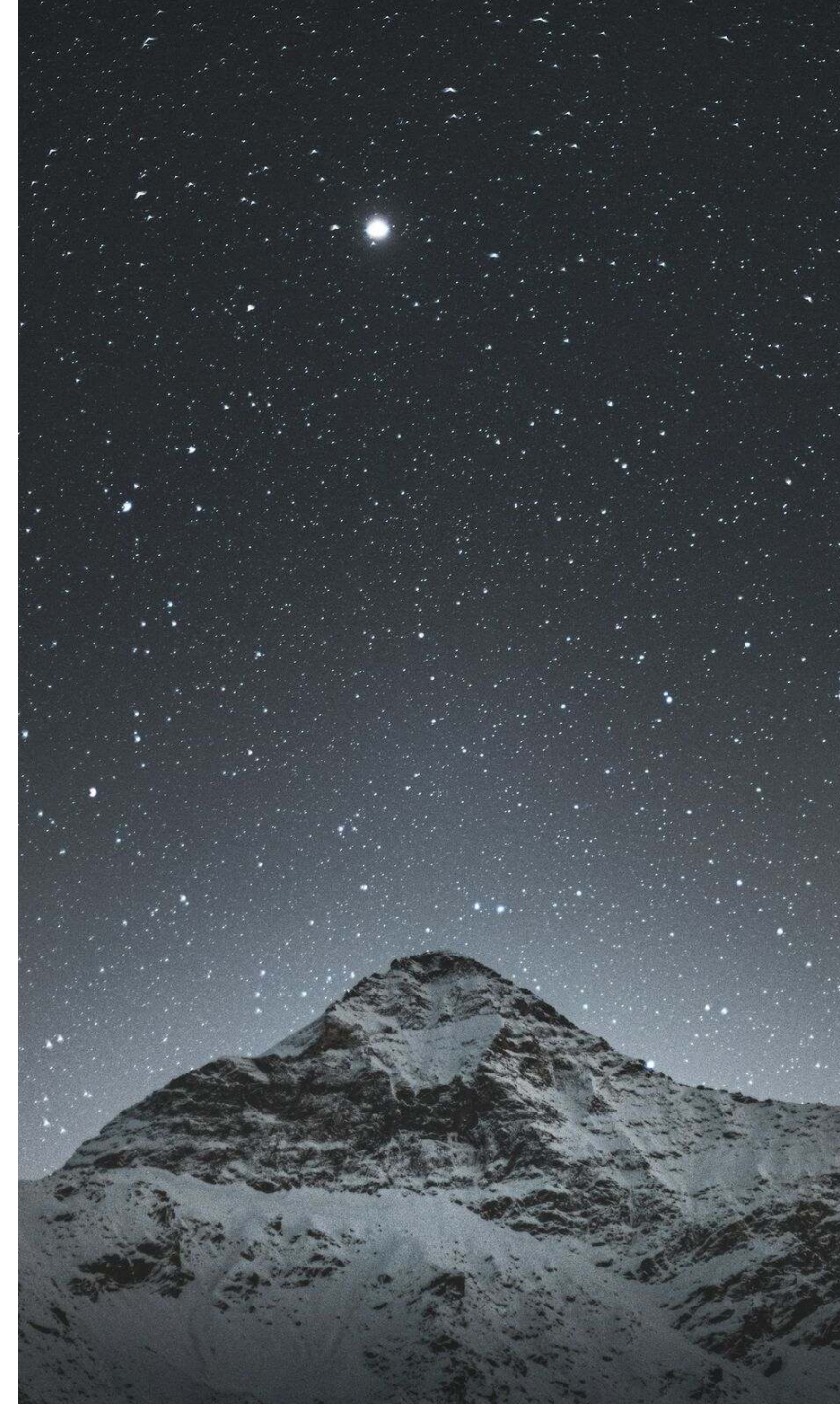
Mein Nordstern

Stell dir vor, du siehst diesen Pull Request an einem Freitagnachmittag:



Wie zuversichtlich bist du, dieses Spring Boot Upgrade zu mergen und in Produktion zu deployen, sobald die Pipeline grün ist?

Gute Tests finden nicht nur Bugs – Tests (plus Automatisierung) geben dir das Vertrauen, ohne Zögern zu deployen.



Ziele für die nächsten 45 Minuten

- Das Fundament für erfolgreiches Testen von Spring Boot Anwendungen legen
- Einführung in Spring Boots hervorragende Test-Unterstützung
- Spring Boot Testing Best Practices und häufige Pitfalls
- Hands-On Tipps zur Optimierung von Build-Zeiten



Über Philip

- Selbstständiger Entwickler aus Herzogenaurach 🍺
- Blogging & Content-Erstellung mit Fokus auf das Testen von Java und speziell Spring Boot Anwendungen 🌿
- Gründer der PragmaTech GmbH - Entwickler befähigen, Software häufiger und mit mehr Vertrauen zu deployen 🚢



Agenda

- Einführung
- Testen mit Spring Boot
 - Part 1: Die Spring Boot Test Pyramide
 - Part 2: Geschwindigkeit & Stabilität für deine Spring Boot Test Suite
 - Part 3: Warum & wann Spring Boot Tests Probleme machen
- Zusammenfassung & Ausblick
- FAQ



Part 1: Die Spring Boot Test Pyramide

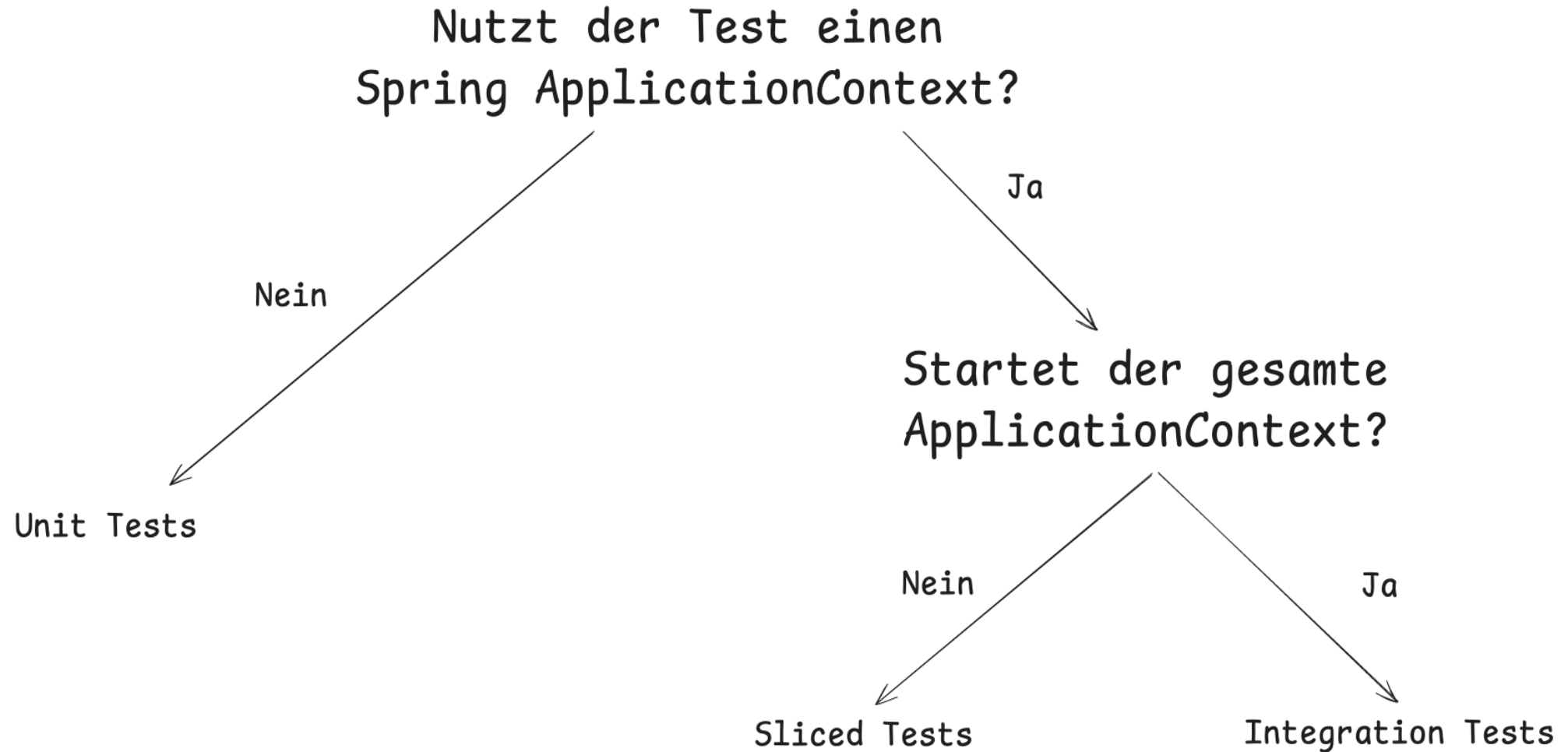


Testing - Pyramid, Honeycomb, Diamond, Trophy?

- Die klassische Testpyramide ist ein guter Ausgangspunkt, aber kein Dogma
- Nicht 1:1 auf jedes Projekt übertragbar
- Viele alternative Modelle: Testing Trophy, Testing Honeycomb, Testing Diamond, etc.
- Die richtige Teststrategie hängt stark vom Projektkontext ab
- Schwer messbar, aber entscheidend: das subjektive Selbstvertrauen bei der Entwicklung & Deployment



Spring Boot Testarten



Unit Testing mit Spring Boot



Unit Testing 101

- Spring Boot Starter Test ("Testing Schweizer Taschenmesser"): Bringt notwendige Test-Bibliotheken mit (JUnit, Mockito, AssertJ, etc.)
- Abhängigkeiten von außen bereitstellen (Dependency Injection)
- Kleine Klassen/Methode mit Single Responsibility entwickeln
- Nur die öffentliche API der Klasse/Methode testen
- Verhalten prüfen, nicht Implementierungsdetails
- TDD kann helfen, (bessere) Klassen zu entwerfen



Unit Testing Beispiel

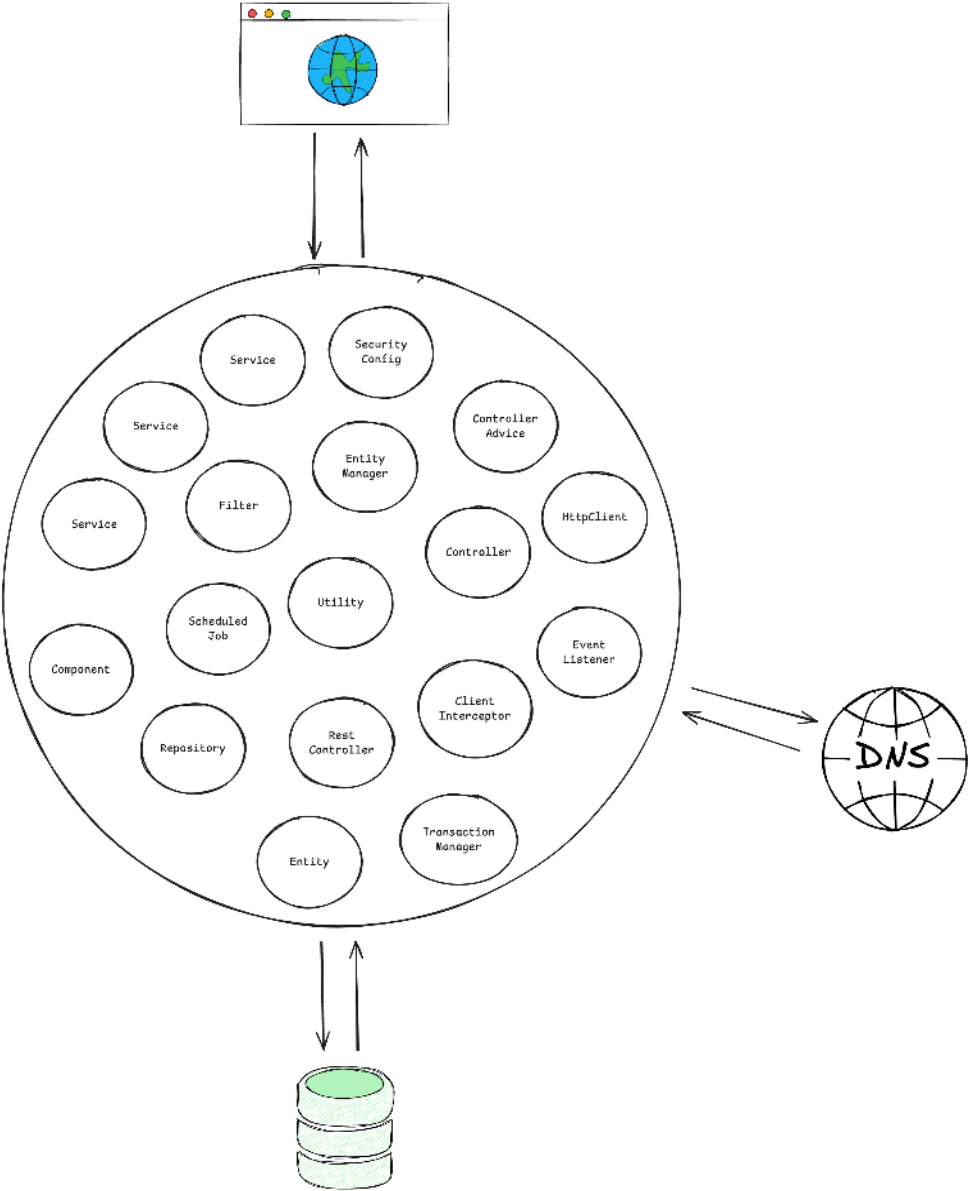
```
1 @ExtendWith(MockitoExtension.class)
2 class CustomerServiceTest {
3
4     @Mock
5     private CustomerRepository customerRepository;
6
7     @InjectMocks
8     private CustomerService customerService;
9
10    @Test
11    void shouldCreateNewCustomerWhenNameDoesNotExist() {
12
13        when(customerRepository.findByCustomerName("duke"))
14            .thenReturn(empty());
15
16        when(customerRepository.save(any(CustomerEntity.class)))
17            .thenAnswer(invocation -> {
18                CustomerEntity storedCustomer = invocation.getArgument(0);
19                storedCustomer.setId("42");
20                return storedCustomer;
21            });
22
23        String customerId = customerService.createNewCustomer("duke");
24
25        assertThat(customerId).isEqualTo("42");
26    }
27 }
```



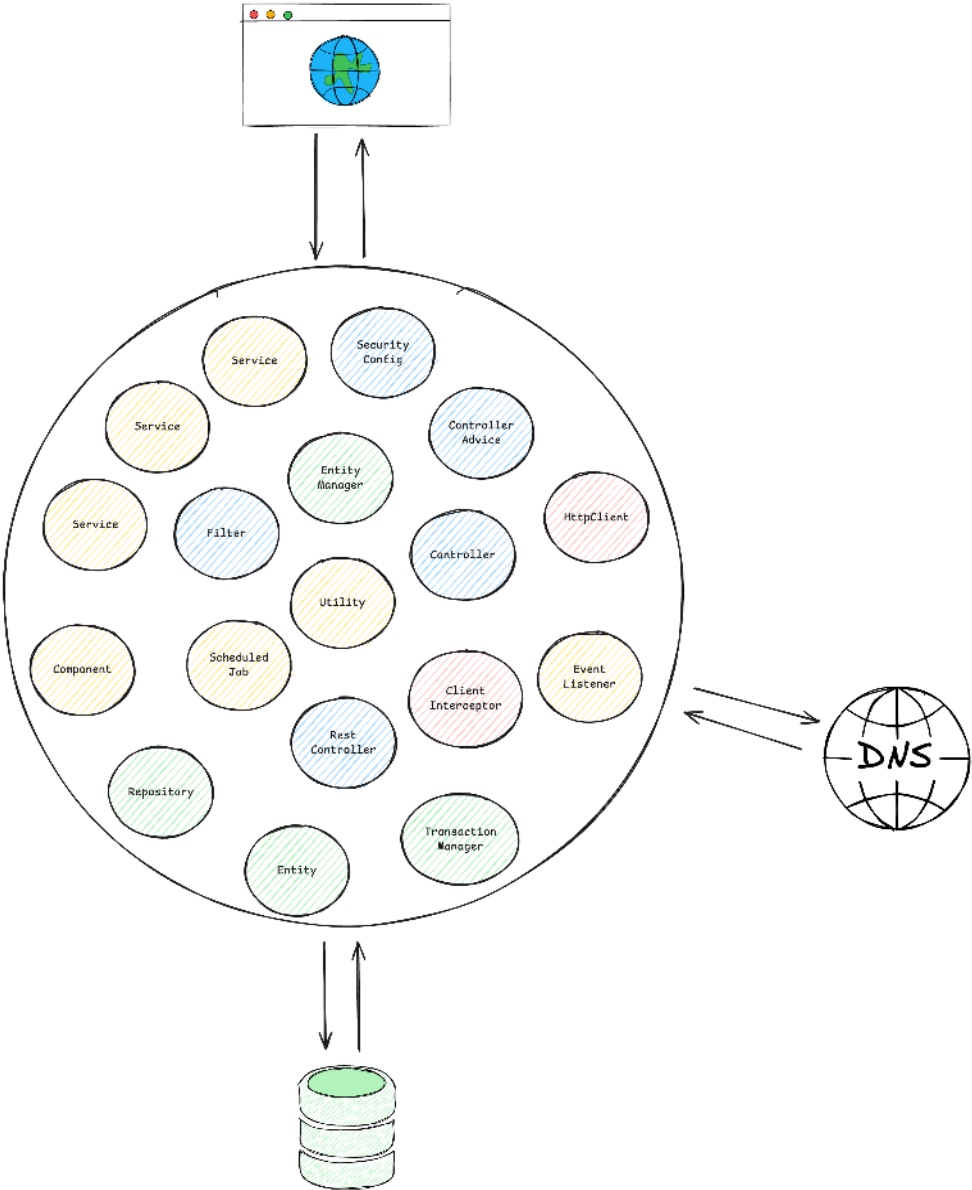
Sliced Testing mit Spring Boot

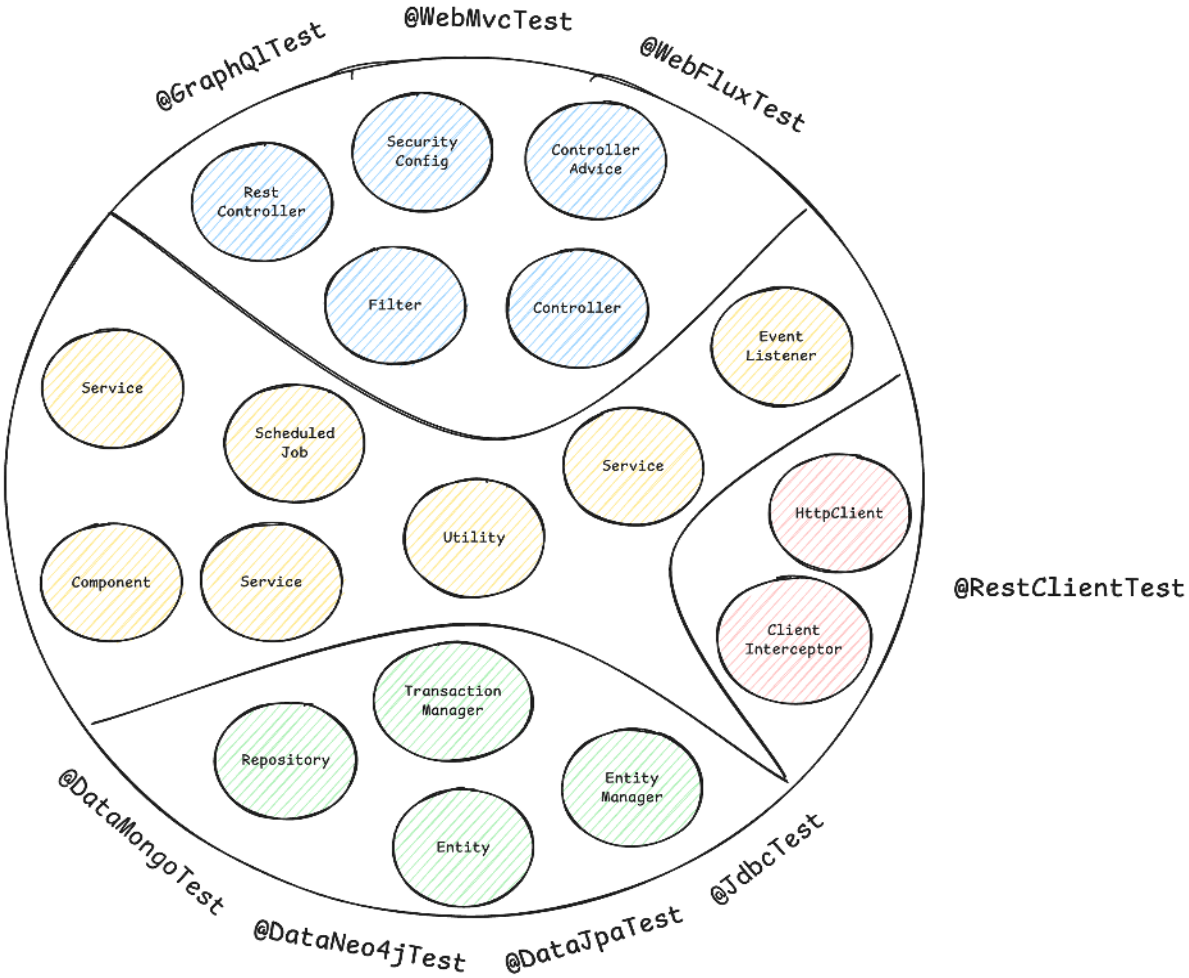


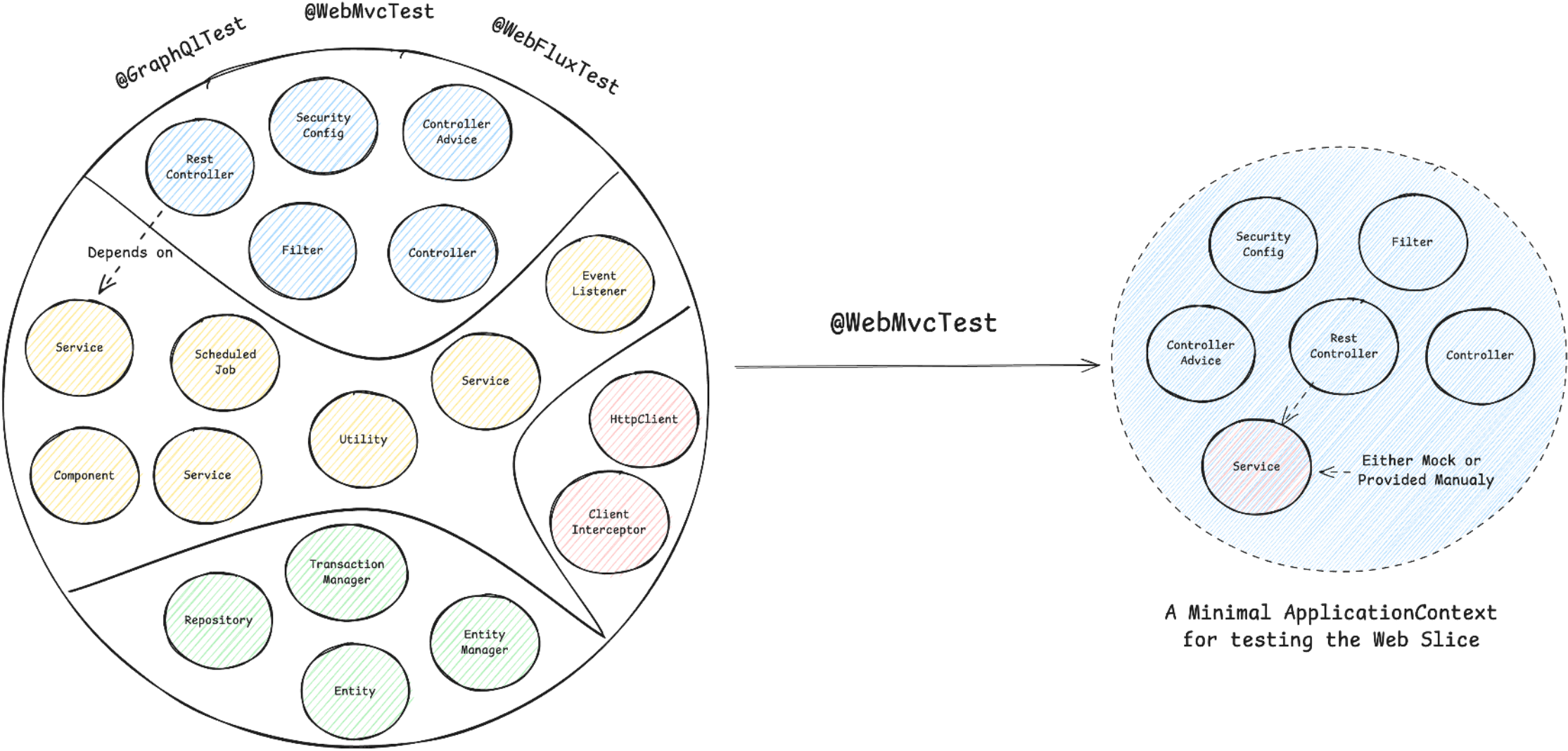
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext







A Minimal ApplicationContext
for testing the Web Slice



Spring Boot Test Slice Beispiel: @WebMvcTest

```
1 @WebMvcTest(CustomerController.class)
2 @Import(SecurityConfig.class)
3 class CustomerControllerTest {
4
5     @Autowired
6     private MockMvc mockMvc;
7
8     @MockitoBean
9     private CustomerService customerService;
10
11     @Test
12     @WithMockUser
13     void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14         // ...
15     }
16 }
```



Integration Testing mit Spring Boot



Integration Testing Herausforderungen

- Externe Infrastruktur mit Testcontainers bereitstellen
- Servlet Container (z.B. Tomcat) starten mit: `@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)`
- WireMock/MockServer für das Stubben externer HTTP-Services in Betracht ziehen
- Controller-Endpunkte testen via: `MockMvc`, `WebTestClient`, `TestRestTemplate`



Testcontainers 101

Infrastrukturkomponenten (Datenbanken, Message Broker, etc.) in Docker-Containern für unsere Tests zu betreiben wird mit **Testcontainers** kinderleicht:

```
1 @Container
2 @ServiceConnection
3 static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withUsername("test")
6     .withPassword("test")
7     .withInitScript("init-postgres.sql");
```

Das gibt uns eine kurzlebige PostgreSQL-Datenbank für unsere Tests:

```
1 $ docker ps
2 CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
3 a958ee2887c6   postgres:16-alpine   "docker-entrypoint.s..." 10 seconds ago Up 9 seconds  0.0.0.0:32776->5432/tcp, [::]:32776->5432/tcp affectionate_cannon
4 ad0f804068dc   testcontainers/ryuk:0.12.0 "/bin/ryuk"              10 seconds ago Up 9 seconds  0.0.0.0:32775->8080/tcp, [::]:32775->8080/tcp testcontainers-ryuk-1f9f76a6-46d4-4e19-85c1-e8364da12804
```

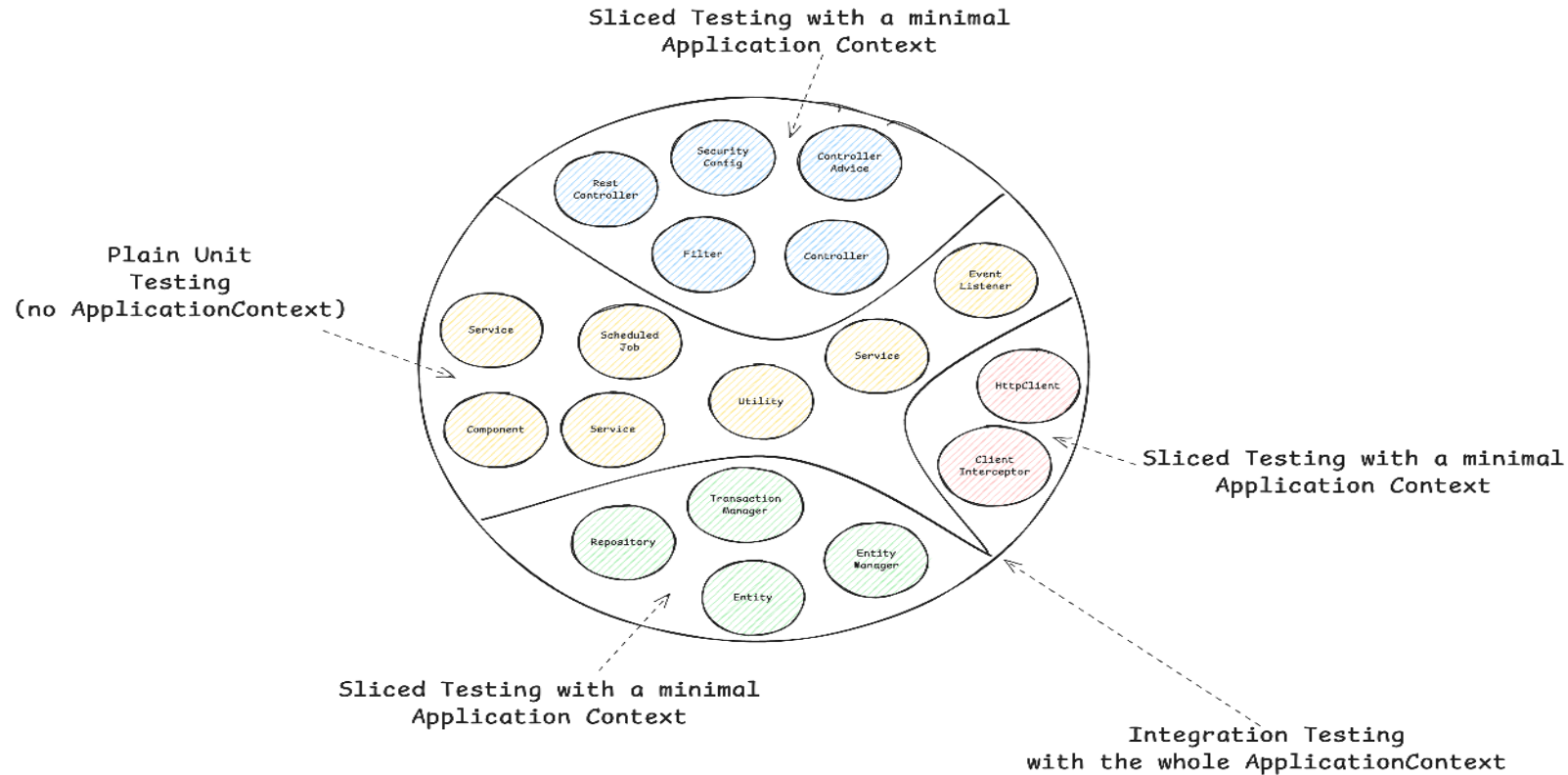


Integration Test Beispiel

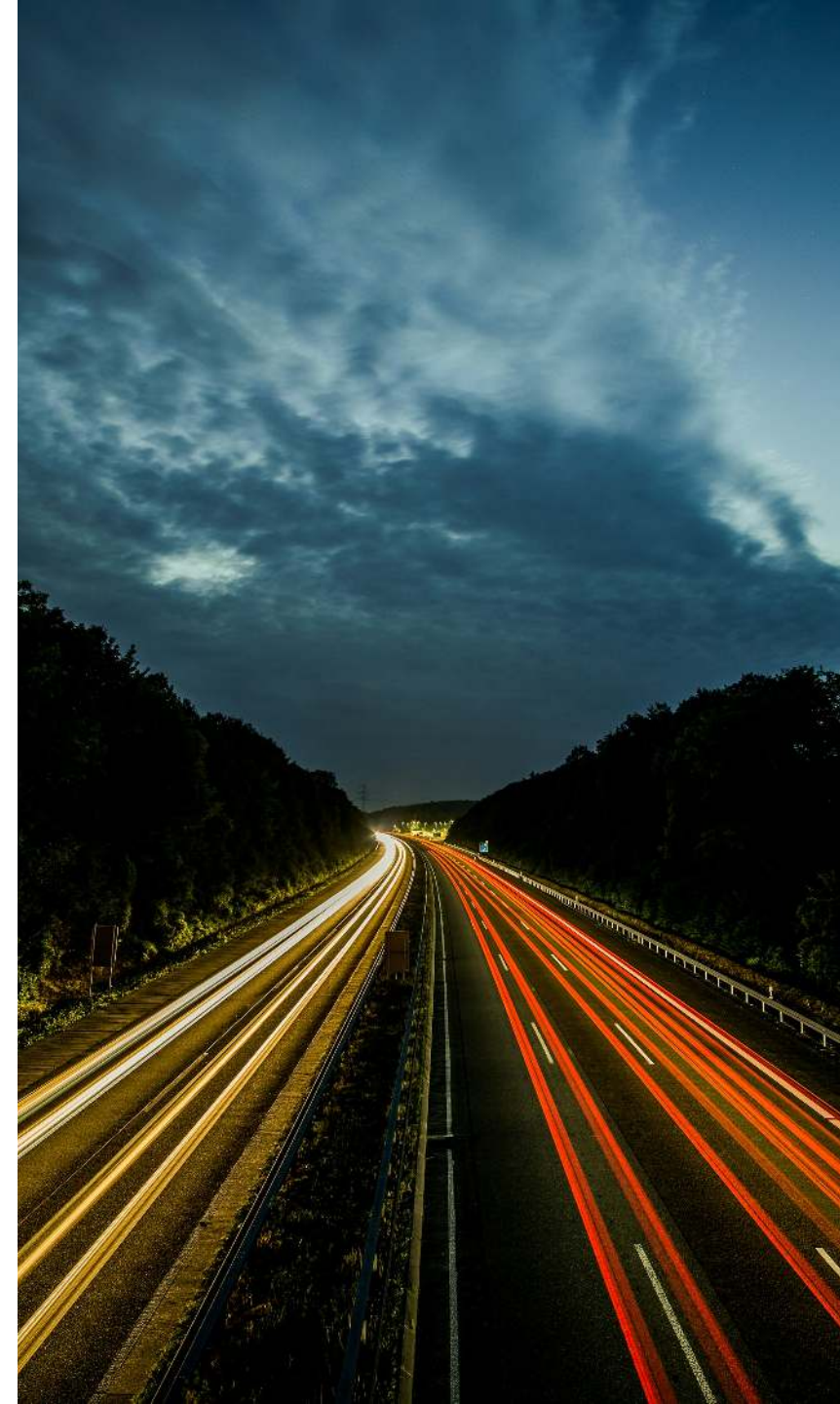
```
1 @AutoConfigureWebTestClient // Spring Boot 4.0
2 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
3 class ApplicationServletContainerIT {
4
5     @LocalServerPort
6     private int port; // <-- we're running on a real port
7
8     @Test
9     void contextLoads(@Autowired WebTestClient webTestClient) {
10         webTestClient
11             .get()
12             .uri("/api/customers")
13             .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
14             .exchange()
15             .expectStatus()
16             .isOk();
17     }
18 }
```



Zusammenfassung Part 1



Part 2: Geschwindigkeit & Stabilität für deine Spring Boot Test Suite



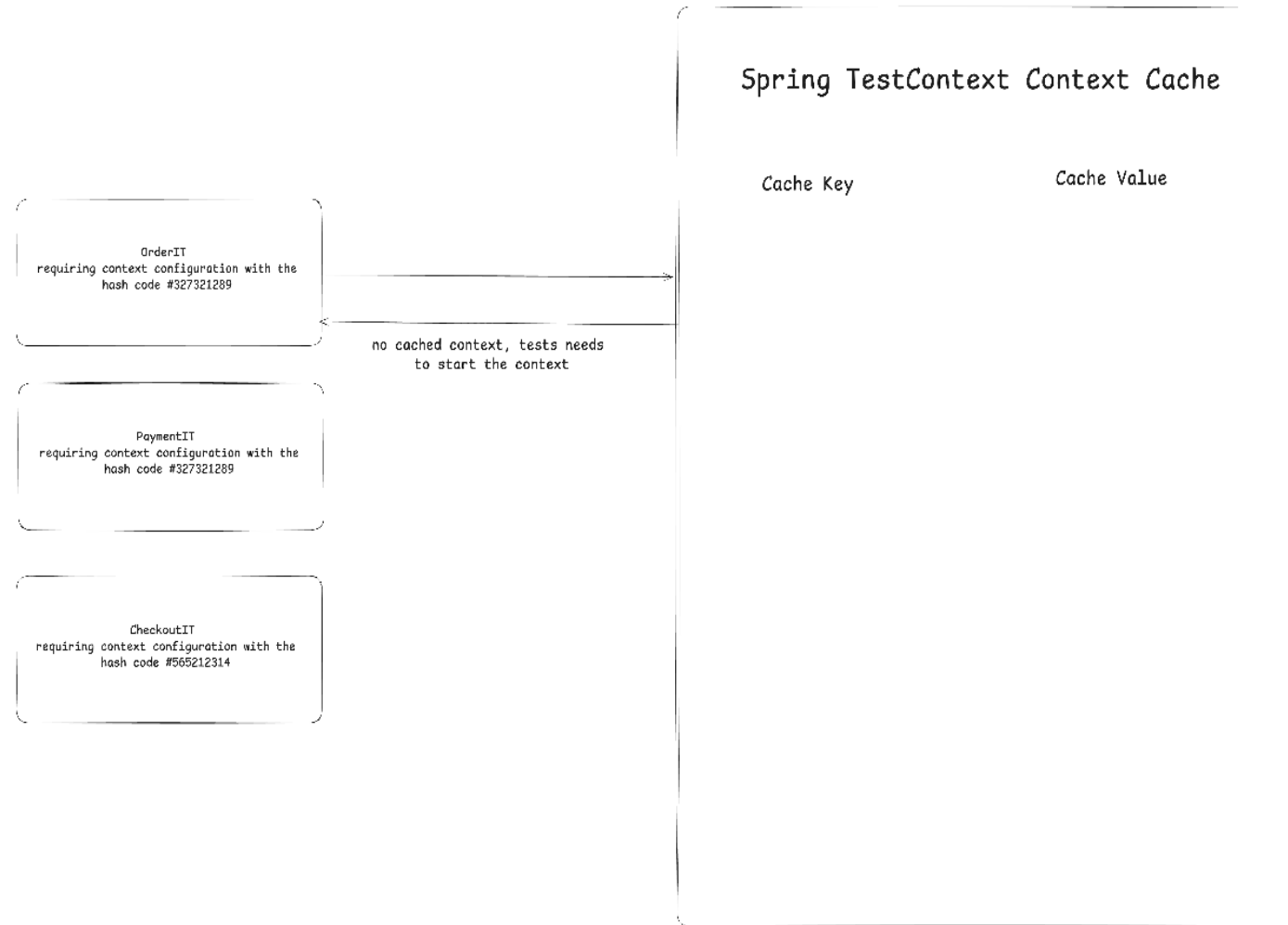
Geschwindigkeitstipp #1 - Spring Context Caching

- **Das Problem:** Integrationstests benötigen einen gestarteten & initialisierten Spring `ApplicationContext`, dessen Start Zeit kostet
- **Die Lösung:** Spring Test `TestContext` Caching – speichert einen bereits gestarteten Spring `ApplicationContext` zur Wiederverwendung
- Dieses Feature ist Teil von Spring Test (in jedem Spring Boot Projekt via `spring-boot-starter-test` enthalten)

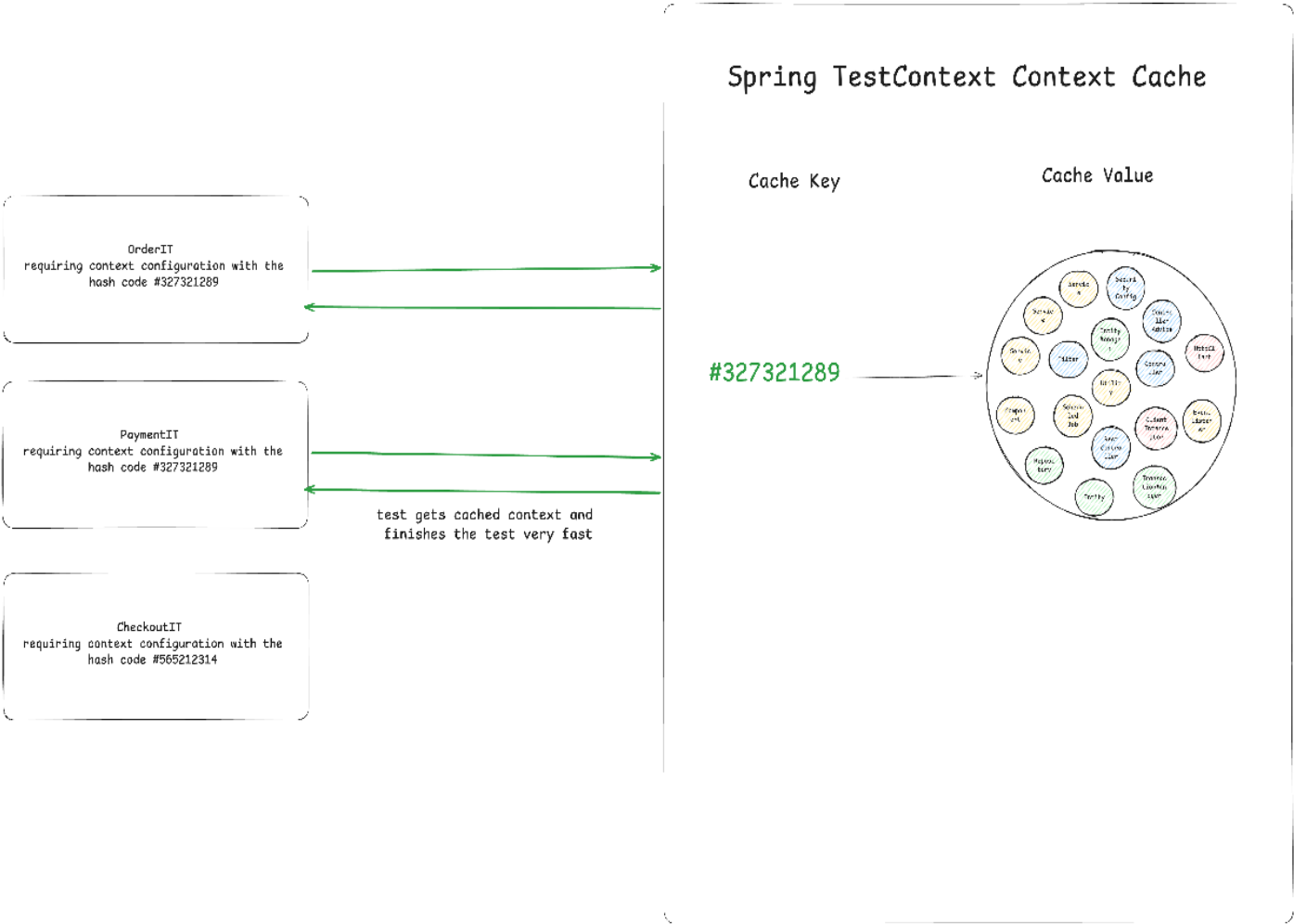
Beispiel für Geschwindigkeitsverbesserung:



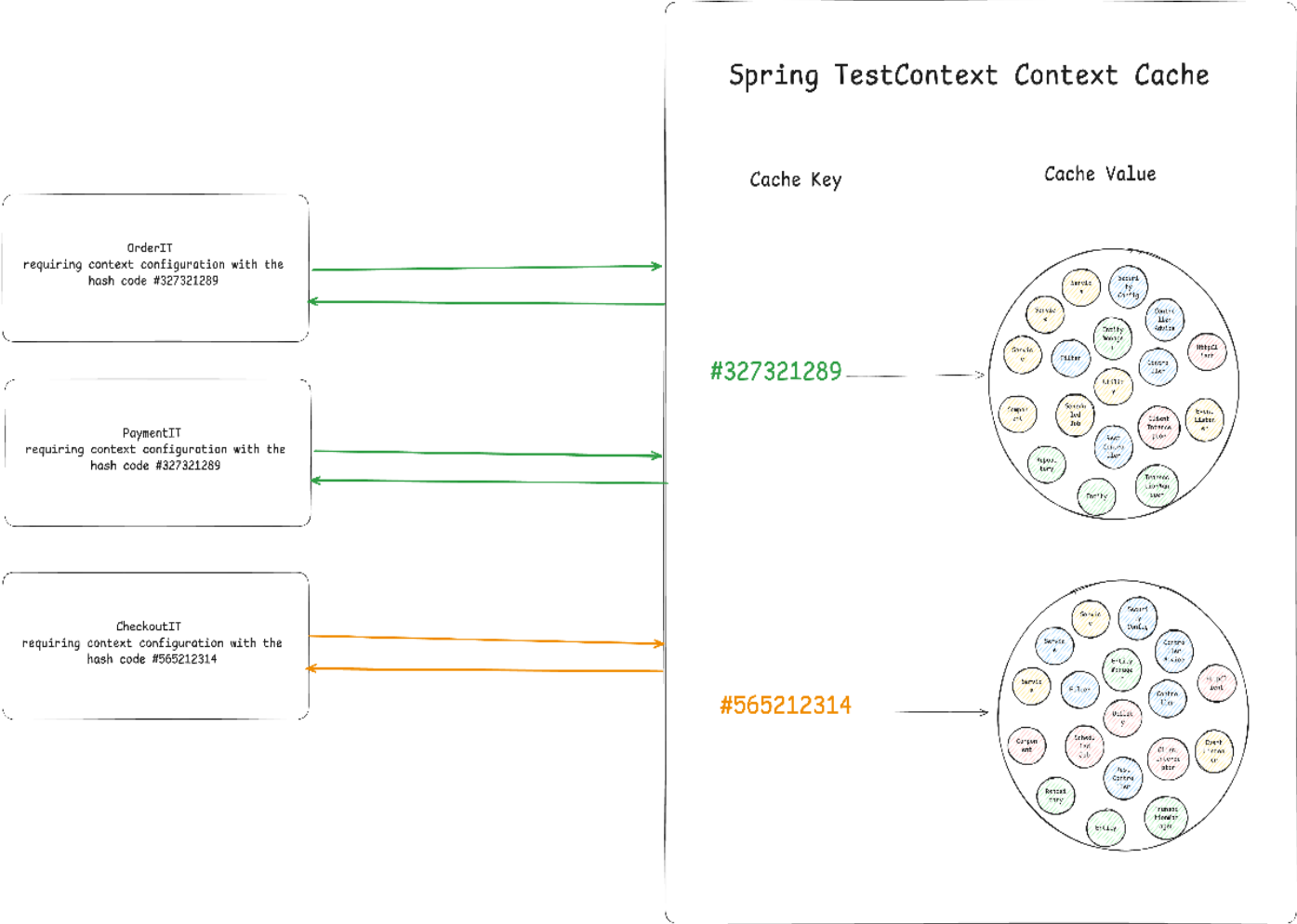
Wie der Cache funktioniert: Schritt 0



Wie der Cache funktioniert: Schritt 1



Wie der Cache funktioniert: Schritt 2



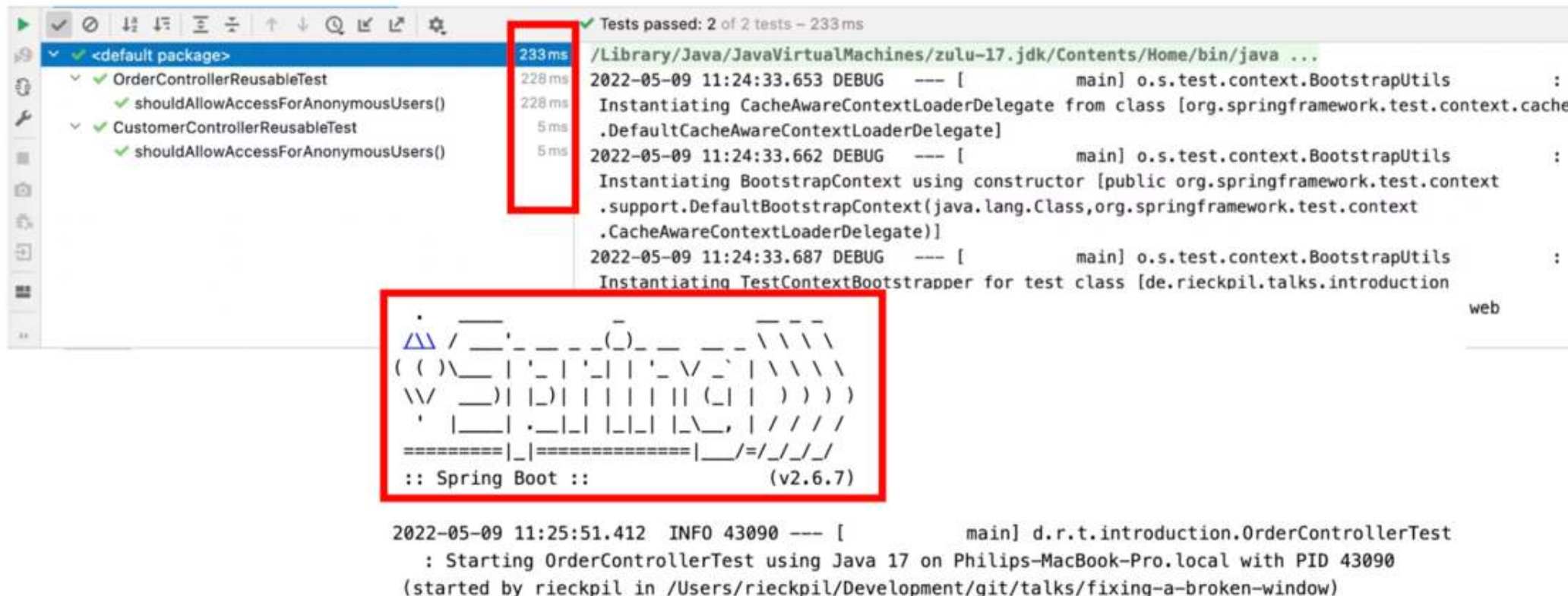
Wie der Cache Key aufgebaut wird

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LruCache(32, 0.75f));
```

Folgendes fließt in den Cache Key (`MergedContextConfiguration`) ein:

- `activeProfiles` (`@ActiveProfiles`)
- `contextInitializersClasses` (`@ContextConfiguration`)
- `propertySourceLocations` (`@TestPropertySource`)
- `propertySourceProperties` (`@TestPropertySource`)
- `contextCustomizer` (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.





Context Restarts erkennen - Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] org.springframework.test.context.cache : Spring test  
ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,  
hitCount = 11, missCount = 1]
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] tractDirtiesContextTestExecutionListener : After test class:  
context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException  
= [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =  
'{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',  
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context  
.SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context  
.filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json  
.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```



Context Restarts erkennen - Tooling



Ein [Open-Source Spring Test Utility](#), das Visualisierung und Einblicke in die Spring Test Ausführung bietet, mit Fokus auf Spring Context Caching Statistiken.

Ziel: Optimierungspotenziale in deiner Spring Test Suite identifizieren, um Builds zu beschleunigen und schneller sowie mit mehr Vertrauen in Produktion zu deployen.



Der Endgegner

Entwickler neigen dazu, bei Integrationstestproblemen AI/StackOverflow zu konsultieren und kopieren oft Ratschläge aus dem Internet, ohne die Auswirkungen zu kennen:

```
1 @SpringBootTest
2 @DirtiesContext
3 // this instructs Spring to remove the context from the cache
4 // and rebuild a new context on every request
5 public abstract class AbstractIntegrationTest {
6
7 }
```

Das Setup oben deaktiviert das Context-Caching-Feature und verlangsamt die Builds erheblich!



Ausblick Spring Framework 7: Pausing von Contexts

Siehe Release Notes von [Spring Framework 7.0.0 M7](#).

Pausing of Test Application Contexts

The Spring TestContext framework is caching application context instances within test suites for faster runs. As of Spring Framework 7.0, we now pause test application contexts when they're not used.

This means an application context stored in the context cache will be stopped when it is no longer actively in use and automatically restarted the next time the context is retrieved from the cache.

Specifically, the latter will restart all auto-startup beans in the application context, effectively restoring the lifecycle state.



Geschwindigkeitstipp #2 - Test Parallelisierung

Anforderungen:

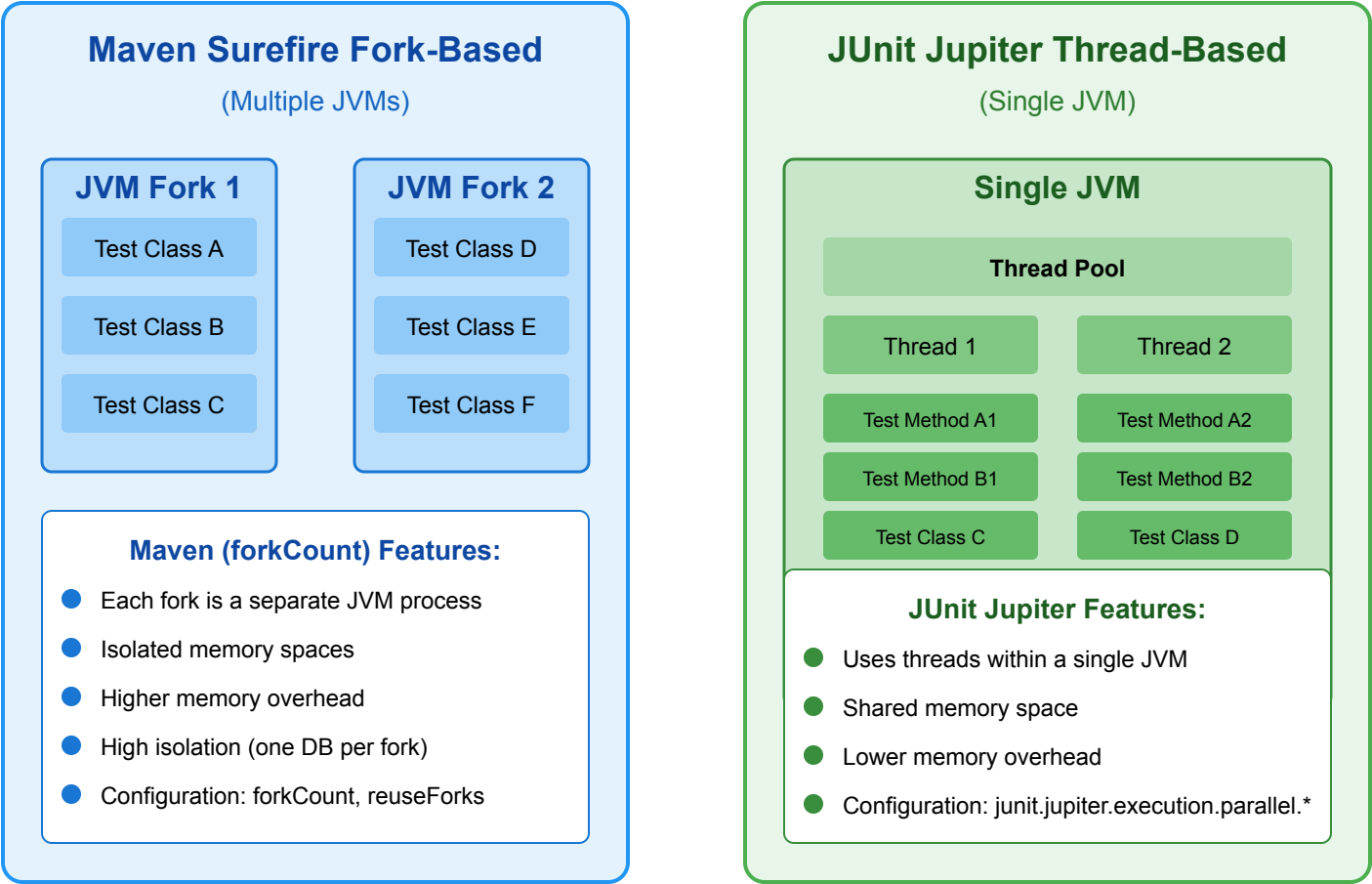
- Kein shared State
- Keine Abhängigkeit zwischen Tests und ihrer Ausführungsreihenfolge
- Keine Mutation von globalem State

Zwei Wege, um das zu erreichen:

- Neue JVM mit Surefire/Failsafe forken und parallel laufen lassen -> mehr Ressourcen, aber isolierte Ausführung
- JUnit Jupiters Parallelisierungsmodus nutzen und in derselben JVM mit mehreren Threads laufen lassen



Java Test Parallelization Options



Geschwindigkeitstipp #3 - AI & Tools

- [Diffblue Cover](#): AI-Agent für Unit Testing von komplexem (Spring Boot) Java-Code im großen Maßstab
- Mein bevorzugter CLI Code Agent: Claude Code
- TDD mit einem LLM
- (Nicht AI, aber trotzdem nützlich) OpenRewrite für [automatische Code-Migrationen](#) (z.B. JUnit 4 -> JUnit 5 -> JUnit 6)
- Anforderungen klar definieren in z.B. `claude.md` oder Cursor-Rule-Dateien, um eine einheitliche Teststruktur zu etablieren



Geschwindigkeitstipp #4 - Testcontainers Container wiederverwenden

- Wenn möglich Container-Reuse in Testcontainers aktivieren: `.withReuse(true)`
- Singleton-Container pro Testlauf eignen sich besser als `@Testcontainers` (Container pro Testklasse)
- Containerstart beschleunigen durch z.B. vordefinierten Datenbank-Snapshots

```
1 private static PostgreSQLContainer<?> postgresModule = new PostgreSQLContainer<>("myteampostgres:42")
2   .withDatabaseName("testdb")
3   .withUsername("testuser")
4   .withPassword("testpass");
5
6 static {
7     postgresModule.start();
8 }
```



Part 3: Warum & wann Spring Boot Tests Probleme machen

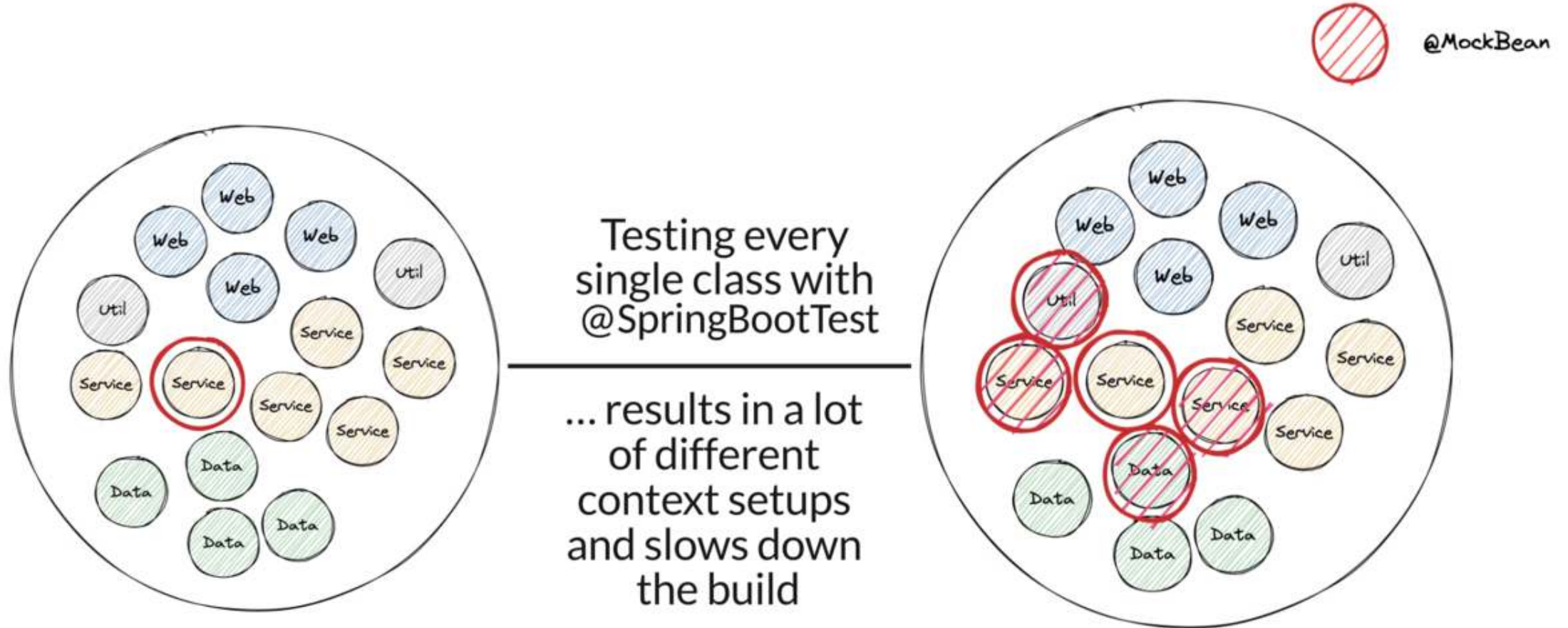


Pitfall 1: `@SpringBootTest` "Besessenheit"

- Der Name könnte suggerieren, es sei eine Universallösung, aber das ist es nicht
- Es hat seinen Preis: der (gesamte) Application Context wird gestartet
- Nützlich für Integrationstests, die die ganze Anwendung verifizieren, aber nicht zum Testen einzelner Services in Isolation
- Mit Unit Tests starten, prüfen ob Sliced Tests anwendbar sind und erst dann `@SpringBootTest` nutzen



@SpringBootTest "Besessenheit" Visuell



Pitfall 2: @MockitoBean vs. @MockBean vs. @Mock

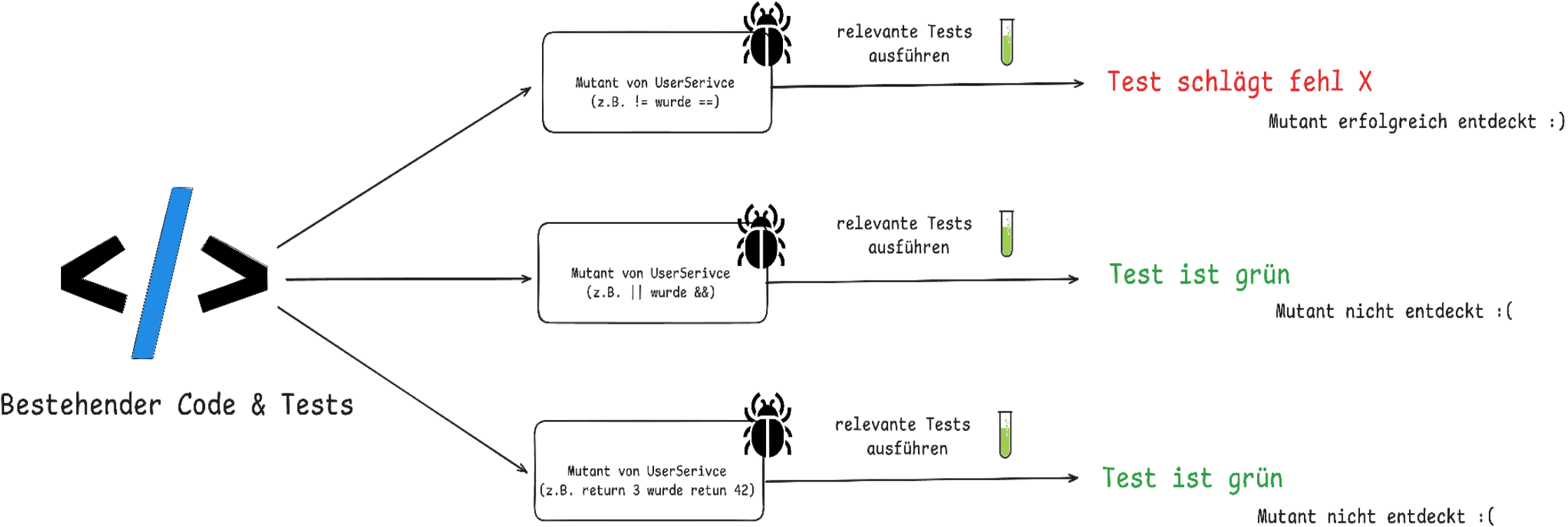
- @MockBean ist eine Spring Boot spezifische Annotation, die eine Bean im Application Context durch ein Mockito Mock ersetzt
- @MockBean ist zugunsten der neuen @MockitoBean Annotation deprecated
- @Mock ist eine Mockito Annotation, nur für Unit Tests
- Golden Mockito Rules:
 - Do not mock types you don't own
 - Don't mock value objects
 - Don't mock everything
 - Show some love with your tests



Pitfall 3: Code Coverage Illusion

- Hohe Code Coverage kann ein **falsches Gefühl von Sicherheit** vermitteln
- Mutation Testing mit **PIT**
- Über Line Coverage hinaus: Traditionelle Tools wie JaCoCo zeigen, welcher Code während der Tests ausgeführt wird, aber PIT verifiziert, ob unsere Tests tatsächlich erkennen, wenn Code sich falsch verhält, indem "**Mutationen**" in unseren Quellcode eingeführt werden.
- Qualitätsgarantie: PIT **modifiziert unseren Code** automatisch (ändert Bedingungen, Rückgabewerte, etc.), um sicherzustellen, dass unsere Tests fehlschlagen, wenn sie sollten, und **deckt blind Spots** in scheinbar umfassenden Test Suites auf.





Testing Pitfall 4: JUnit 4 vs. JUnit 5 (vs. JUnit 6)

- Man kann beide Versionen im selben Projekt mischen, aber nicht in derselben Testklasse
- Beim Durchsuchen des Internets (aka. StackOverflow/Blogs/LLMs) nach Lösungen findet man möglicherweise Test-Setups, die noch für JUnit 4 sind
- Schnell das falsche `@Test` importiert und schon verschwendet man eine Stunde, weil der Spring Context nicht wie erwartet funktioniert



JUnit 4	JUnit 5
@Test from org.junit	@Test von org.junit.jupiter.api
@RunWith	@ExtendWith/@RegisterExtension
@ClassRule/@Rule	-
@Before	@BeforeEach
@Ignore	@Disabled
@Category	@Tag



Zusammenfassung & Ausblick

- Spring und Spring Boot bieten viele hervorragende Testing-Features
- Java bietet ein ausgereiftes & umfangreiches Testing-Ökosystem
- Das **Context-Caching-Feature** für schnelle Builds berücksichtigen
- **Sliced Testing** hilft, isolierte Tests mit minimalem Kontext zu schreiben
- Build-Zeiten durch Test Parallelisierung & den richtigen Einsatz von Testcontainers reduzieren
- Viele neue Testing-Features sind Teil neuer Releases



Weitere Spring Boot Testing Angebote

- Online Kurs: [Testing Spring Boot Applications Masterclass](#) (on-demand, 12 Stunden, 130+ Module)
- eBook: [30 Testing Tools and Libraries Every Java Developer Must Know](#)
- eBook: [Stratospheric - From Zero to Production with AWS](#)
- Spring Boot [testing workshops](#) (vor Ort/remote/hybrid)
- [Consulting Angebote](#), z.B. das Test Maturity Assessment für Projekte/Teams



Ergänzendes Spring Boot Testing eBook

- Hol dir das ergänzende Spring Boot Testing eBook **kostenlos** (statt \$9)
- 120+ Seiten mit praktischen Hands-on-Tipps, um Code mit Vertrauen zu deployen
- Hol dir das eBook, indem du dich über den QR-Code auf der nächsten & letzten Folie für unseren [Newsletter](#) anmeldest

TESTING SPRING BOOT APPLICATIONS DEMYSTIFIED



Philip Riecks



Viel Erfolg beim Testen!

Hol dir dein kostenloses Spring Boot Testing eBook:



Melde dich jederzeit via:

- [LinkedIn](#) (Philip Riecks)
- [X](#) (@rieckpil)
- [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

